

Lab: App-Tier Push Plugin That Restricts Access at the Web Tier

You will build a two-tier Tomcat setup where the **app tier decides** which of its hosted paths are public, generates an allow-list, and **pushes** it to a **web-tier** Tomcat. The web tier forwards only approved paths to the app tier and blocks everything else, even paths the app tier still hosts.

Everything runs on one Linux machine. The two Tomcats are separated by port.

What each tier does

- **App tier (port 9080).** Hosts the real endpoints. Declares which paths are public, generates the allow-list from that declaration, and pushes it to the web tier on startup. A push plugin (`AllowListPublisher`) does the pushing.
- **Web tier (port 8080).** The front door. Receives the pushed allow-list, stores it, and on every request forwards approved paths to the app tier or returns 403 for anything else. A servlet filter (`GatewayFilter`) is the gate.

The app tier is the source of truth. The web tier only enforces what it was told.

Port map (all on 127.0.0.1)

Instance	HTTP port	Shutdown port	Role
Tomcat web	8080	8005	gateway, enforces allow-list
Tomcat app	9080	9005	hosts endpoints, pushes list

Endpoints on the app tier

Path	Public?	In pushed list?	Reachable via web tier?
<code>/api/products</code>	yes	yes	yes
<code>/api/orders</code>	yes	yes	yes
<code>/api/internal/metrics</code>	no	no	no (blocked at 8080)

`/api/internal/metrics` is the proof case. The app tier hosts it and answers on 9080, but because it is never pushed, the web tier blocks it on 8080.

1. Prerequisites

- One Linux machine.

- Tomcat 10.1 at `/opt/tomcat` (`CATALINA_HOME`). Tomcat 9 needs the `web.xml` namespace changed to `javaee` and the `jakarta.*` imports changed to `javax.*`.
- JDK 17 or later (the code uses `java.net.http.HttpClient`).
- `curl` for testing.

2. Create two Tomcat instances from one install

```
export CATALINA_HOME=/opt/tomcat

for n in webtier apptier; do
  mkdir -p /opt/tomcat/$n/{conf,logs,temp,webapps,work,bin}
  cp -r $CATALINA_HOME/conf/* /opt/tomcat/$n/conf/
done
```

Set the app tier ports so they do not clash with the web tier. Edit

`/opt/tomcat/apptier/conf/server.xml` :

```
<Server port="9005" shutdown="SHUTDOWN">
```

```
<Connector port="9080" protocol="HTTP/1.1"
  connectionTimeout="20000"
  address="127.0.0.1"
  redirectPort="9443"/>
```

Leave the web tier at the defaults (8080 / 8005), and bind it to loopback:

```
<Connector port="8080" protocol="HTTP/1.1"
  connectionTimeout="20000"
  address="127.0.0.1"
  redirectPort="8443"/>
```

Delete or comment out the AJP connector in both files.

3. The web tier: gateway that enforces the allow-list

Three classes make up the web-tier webapp, deployed as ROOT so paths map 1:1.

`AllowListStore` holds the current list and answers the allow decision. An exact match or any child under an approved prefix passes.

```

public boolean allows(String requestPath) {
    for (String entry : paths) {
        if (requestPath.equals(entry)
            || requestPath.startsWith(entry + "/")) {
            return true;
        }
    }
    return false;
}

```

`AllowListServlet` (mapped to `/admin/allowlist`) receives the push. POST replaces the list and is protected by a shared token. GET lists the current allow-list for inspection.

```

String supplied = req.getHeader("X-Push-Token");
if (!token().equals(supplied)) {
    resp.sendError(HttpServletResponse.SC_FORBIDDEN, "bad push token");
    return;
}

```

`GatewayFilter` (mapped to `/*`) is the gate. It lets the admin endpoint through, blocks any path not in the list, and proxies approved paths to the app tier.

```

if (!AllowListStore.get().allows(path)) {
    resp.setStatus(HttpServletResponse.SC_FORBIDDEN);
    resp.getWriter().println("blocked at web tier: " + path);
    return;
}
// approved: forward to backendBase + path

```

The web-tier `web.xml` sets the shared token and the app-tier address:

```

<context-param>
  <param-name>pushToken</param-name>
  <param-value>s3cr3t-web-tier</param-value>
</context-param>
<context-param>
  <param-name>backendBase</param-name>
  <param-value>http://127.0.0.1:9080</param-value>
</context-param>

```

Full source is in `web-tier/`.

4. The app tier: endpoints plus the push plugin

The app tier hosts three servlets (`/api/products`, `/api/orders`, `/api/internal/metrics`) and declares its public paths in `web.xml`:

```

<context-param>
  <param-name>publicPaths</param-name>
  <param-value>/api/products,/api/orders</param-value>
</context-param>
<context-param>
  <param-name>webTierPushUrl</param-name>
  <param-value>http://127.0.0.1:8080/admin/allowlist</param-value>
</context-param>
<context-param>
  <param-name>pushToken</param-name>
  <param-value>s3cr3t-web-tier</param-value>
</context-param>

```

`AllowListPublisher` is the push plugin. It is a `ServletContextListener`, so it runs on app-tier startup. It generates the allow-list from `publicPaths` and POSTs it to the web tier with the token.

```

@WebListener
public class AllowListPublisher implements ServletContextListener {
    public void contextInitialized(ServletContextEvent ev) {
        // generate list from publicPaths, POST to webTierPushUrl
    }
}

```

`PublishServlet` (mapped to `/admin/publish`) re-pushes on demand, which you use if the app tier started before the web tier, or after you change `publicPaths`.

Full source is in `app-tier/`.

5. Build both WARs

Each tier has a `build.sh` that compiles against your Tomcat libraries and produces `ROOT.war`.

```

export CATALINA_HOME=/opt/tomcat

cd web-tier && ./build.sh && cd ..
cd app-tier && ./build.sh && cd ..

```

Deploy each as ROOT on its instance:

```

cp web-tier/ROOT.war /opt/tomcat/webtier/webapps/
cp app-tier/ROOT.war /opt/tomcat/apptier/webapps/

```

6. Start in the right order

Start the **web tier first** so the push target exists, then the app tier.

```
CATALINA_BASE=/opt/tomcat/webtier $CATALINA_HOME/bin/catalina.sh start
CATALINA_BASE=/opt/tomcat/apptier $CATALINA_HOME/bin/catalina.sh start
```

Confirm the push landed. The app-tier log records it:

```
grep -i "push plugin" /opt/tomcat/apptier/logs/catalina.out
#   push plugin: pushed to web tier, reply 200 (stored 2 paths)
```

Read the allow-list the web tier now holds:

```
curl http://127.0.0.1:8080/admin/allowlist
#   /api/products
#   /api/orders
```

If the app tier started first, the push failed. Trigger it manually:

```
curl http://127.0.0.1:9080/admin/publish
```

7. Test A: web tier forwards only approved paths

```
curl http://127.0.0.1:8080/api/products
#   app-tier: products list [public]

curl http://127.0.0.1:8080/api/orders
#   app-tier: orders list [public]
```

Both come back from the app tier, proxied through the web tier. The web tier forwarded them because they are in the pushed allow-list.

8. Test B: a hosted-but-unapproved path is blocked at the web tier

First confirm the app tier really does host the metrics endpoint. Hit it directly on 9080:

```
curl -i http://127.0.0.1:9080/api/internal/metrics
#   HTTP/1.1 200
#   app-tier: internal metrics [private, hosted but not approved]
```

Now go through the web tier on 8080:

```
curl -i http://127.0.0.1:8080/api/internal/metrics
#   HTTP/1.1 403
#   blocked at web tier: /api/internal/metrics
```

Same path, two results. The app tier serves it, but the web tier never received it in a push, so the web tier blocks it. This is the core of the exercise.

9. Test C: the app tier controls access dynamically

Change what the app tier considers public and re-push. Edit `/opt/tomcat/apptier/webapps/ROOT/WEB-INF/web.xml`, drop `/api/orders` from `publicPaths` so it reads `/api/products`, then re-push:

```
curl http://127.0.0.1:9080/admin/publish
curl http://127.0.0.1:8080/admin/allowlist
# /api/products

curl -i http://127.0.0.1:8080/api/orders
# HTTP/1.1 403
# blocked at web tier: /api/orders
```

`/api/orders` still exists on the app tier, but the moment the app tier stopped publishing it, the web tier stopped forwarding it. Access is driven entirely by what the app tier pushes.

Editing a deployed `web.xml` reloads the context, so give it a second before re-pushing. For a cleaner demo, keep `publicPaths` in an external file the publisher reads, so no redeploy is needed.

10. Stop everything

```
CATALINA_BASE=/opt/tomcat/apptier $CATALINA_HOME/bin/catalina.sh stop
CATALINA_BASE=/opt/tomcat/webtier $CATALINA_HOME/bin/catalina.sh stop
```

How it maps to the requirements

- **Web tier reads a pushed allow-list and forwards only approved paths.** `GatewayFilter` checks `AllowListStore` and proxies approved paths to 9080.
- **App server generates and pushes the allow-list.** `AllowListPublisher` builds the list from `publicPaths` and POSTs it to the web tier on startup.
- **A path not in the list is blocked at the web tier even if the app hosts it.** `/api/internal/metrics` answers on 9080 but returns 403 on 8080.

Hardening notes for real use

- Replace the shared token with mutual TLS between the tiers, or restrict the push endpoint to the app tier's source IP with a firewall rule.
- Push over HTTPS so the list and token do not cross the wire in clear text.
- Persist the last-pushed list on the web tier (a file), so a web-tier restart does not open a window where nothing is allowed until the next push. Fail closed: an empty list blocks everything, which this lab already does.

- Sign the pushed payload so the web tier can verify it came from the app tier.
- Add allow-list versioning and a health endpoint so you can see which version each web-tier node is enforcing.

Troubleshooting

Symptom	Fix
Every path returns 403 through 8080	No push arrived. Check the app-tier log, then <code>curl http://127.0.0.1:9080/admin/publish</code> .
Push returns 403	Token mismatch. <code>pushToken</code> must match in both <code>web.xml</code> files.
502 from the web tier	App tier is down or on the wrong port. Confirm 9080 answers directly.
Approved path still blocked	Confirm the exact path. Matching is on <code>/</code> -segment prefixes, so <code>/api/products</code> allows <code>/api/products</code> and <code>/api/products/42</code> but not <code>/api/productsX</code> .
Port already in use	App tier must use 9080/9005. Check with <code>ss -ltnp \ grep -E '8080\ 9080'</code> .